Algoritmos de optimización - Seminario

Nombre y Apellidos: Sergio Monrabal Romero y Alex Sanz Morant
Problema:

> 3. Combinar cifras y operaciones

Descripción del problema:(copiar enunciado)

• El problema consiste en analizar el siguiente problema y diseñar un algoritmo que lo resuelva.

• Disponemos de las 9 cifras del 1 al 9 (excluimos el cero) y de los 4 signos básicos de lasoperaciones fundamentales: suma(+), resta(-), multiplicación(*) y división(/)

• Debemos combinarlos alternativamente sin repetir ninguno de ellos para obtener una cantidad dada. Un ejemplo sería para obtener el 4: 4+2-6/3*1 = 4

• Debe analizarse el problema para encontrar todos los valores enteros posibles planteando las siguientes cuestiones:

- ¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?
- ¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo ?

• Nota: Es posible usar la función de python “eval” para evaluar una expresión

(*) La respuesta es obligatoria

(*)¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

¿Cuantas posibilidades hay teniendo en cuenta todas las restricciones.

Respuesta

En el problema se cuenta con dos restricciones, por un lado no se pueden repetir la cifra y por el otro no se puede repetir la operación. Teniendo en cuenta esto las posiblidades que hay son las siguientes:
-La primera cifra se elegirá entre 9 posibles.
-La primera operación se elegirá entre las 4 posibles.
-La segunda cifra se elegirá entre 8 posibles.

-La segunda operación se elegirá entre las 3 posibles.
-La tercera cifra se elegirá entre 7 posibles.
-La tercera operación se elegirá entre las 2 posibles.
-La cuarta cifra se elegirá entre 6 posibles.
-La cuarta operación se elegirá entre las 1 posibles.
-La quinta cifra se elegirá entre 5 posibles.
Así pues, multiplicando las posibles elecciones de cifras y números anterior, tendremos que la cantidad de operaciones posibles será de 9!=362880.

Sin embargo, si no tenemos en cuenta las restricciones anteriores, no disminuirá el número de elecciones de cifra y operador que realicemos, quedando las elecciones de la siguiente manera:
-La primera cifra se elegirá entre 9 posibles.
-La primera operación se elegirá entre las 4 posibles.
-La segunda cifra se elegirá entre 9 posibles.
-La segunda operación se elegirá entre las 4 posibles.
-La tercera cifra se elegirá entre 9 posibles.
-La tercera operación se elegirá entre las 4 posibles.
-La cuarta cifra se elegirá entre 9 posibles.
-La cuarta operación se elegirá entre las 4 posibles.
-La quinta cifra se elegirá entre 9 posibles.
Por lo tanto el número de posiblidades, sin restricciones, será de $9^5 * 4^4$ = 15116544.

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

La estructura de datos que mejor se adapta al problema es la lista. Las listas son un tipo de estructura de datos muy versátil que cuentan con una serie de funciones built-in que son bastante útiles para la resolución de nuestro problema. En particular, las funciones por las cuales hemos decidido usar este tipo de estructura han sido *pop* e *insert*.

La función *pop* nos ha permitido extraer un elemento de la lista y borrarlo de la misma, gracias a lo cual nos hemos podido asegurar de no repetir números u operaciones.

De forma complementaria, la función *insert* nos ha permitido volver a insertar aquellos elementos que incialmente habíamos quitado de la lista para comprobar una posible solución, la cual hemos descartado.

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

El problema que nos ocupa no se trata de ningún problema cuyo objetivo sea la optimización de algún sistema, sino que se trata del análisis y el diseño de un algoritmo que sea capaz de calcular un entero dadas ciertas condiciones. Por este motivo, no podemos decir que exista ninguna función objetivo ya que estas, por definición, están asociadas a un problema de optimización.

Análogamente podemos concluir que no se trata de ningún problema de maximización ni minimización, ya que en ese caso estaríamos ante un problema de optimización.

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

In [ ]:
```
#Fuerza Bruta

#Se definen dos listas con los números y las operaciones
lista_operaciones = ["+","-","*","/"]
lista_numeros = [1,2,3,4,5,6,7,8,9]

def operaciones_paraFB(target):
    nums = lista_numeros
    ops = lista_operaciones

    #Iteramos sobre la lista de números tomando 5 números distintos para comprob
    for num_uno in nums:
        for num_dos in nums:
            if num_dos == num_uno:
                continue
            for num_tres in nums:
                if num_tres == num_uno or num_tres == num_dos:
                    continue
```

```
                for num_cuatro in nums:
                    if num_cuatro == num_uno or num_cuatro == num_dos or num_cua
                        continue
                    for num_cinco in nums:
                        if (num_cinco == num_uno or num_cinco == num_dos or
                            num_cinco == num_tres or num_cinco == num_cuatro):
                            continue

                        #Iteramos sobre la lista de operaciones tomando 4 distin
                        for op_uno in ops:
                            for op_dos in ops:
                                if op_dos == op_uno:
                                    continue
                                for op_tres in ops:
                                    if op_tres == op_uno or op_tres == op_dos:
                                        continue
                                    for op_cuatro in ops:
                                        if op_cuatro == op_uno or op_cuatro == o
                                            continue

                                        #Evaluamos si la expresión con las opera
                                        expr = f"{num_uno}{op_uno}{num_dos}{op_d
                                        if eval(expr) == target:
                                            return expr
    return None #Si no encontramos ninguna combinación devolvemos None

for s in range(-70, 80):
    print(f"{s}: {operaciones_paraFB(s)}")
```

```
-70: None
-69: 1+4/2-8*9
-68: 1+6/2-8*9
-67: 2/1+3-8*9
-66: 2/1+4-8*9
-65: 2/1+5-8*9
-64: 2/1+6-8*9
-63: 2/1+7-8*9
-62: 3/1+7-8*9
-61: 4/1+7-8*9
-60: 1+4/2-7*9
-59: 1+6/2-7*9
-58: 1-7*9+8/2
-57: 2/1+4-7*9
-56: 2/1+5-7*9
-55: 2/1+6-7*9
-54: 3/1+6-7*9
-53: 1+4/2-7*8
-52: 1+6/2-7*8
-51: 1+4/2-6*9
-50: 2/1+4-7*8
-49: 1-6*9+8/2
-48: 2/1+4-6*9
-47: 2/1+5-6*9
-46: 3/1+5-6*9
-45: 1+4/2-6*8
-44: 1-6*8+9/3
-43: 2/1+3-6*8
-42: 1+4/2-5*9
-41: 1-5*9+6/2
-40: 1-5*9+8/2
-39: 1+4/2-6*7
-38: 1-6*7+9/3
-37: 1+4/2-5*8
-36: 1-5*8+6/2
-35: 2/1+3-5*8
-34: 2/1+4-5*8
-33: 1-4*9+6/3
-32: 1+3-8/2*9
-31: 1+4-8/2*9
-30: 1-5*7+8/2
-29: 1-4*8+6/3
-28: 1-4*8+6/2
-27: 1+4/2-5*6
-26: 1-5*6+9/3
-25: 1-4*7+6/3
-24: 1+3-7/2*8
-23: 1+3-6/2*9
-22: 1-3*9+8/2
-21: 1+2-8/3*9
-20: 1+3-6/2*8
-19: 1+4-6/2*8
-18: 1+2-7/3*9
-17: 1+3-6/2*7
-16: 1+3-5/2*8
-15: 1+2-6/3*9
-14: 1+3-4/2*9
-13: 1+2-6/3*8
-12: 1+2-5/3*9
-11: 1+2-6/3*7
```

-10: 1-2*7+9/3
-9: 1+2-4/3*9
-8: 1-2*6+9/3
-7: 1+2*3/6-9
-6: 1+2*3/6-8
-5: 1+2*3/6-7
-4: 1+2/3*6-9
-3: 1+2-3/4*8
-2: 1+2*3/6-4
-1: 1+2-3/6*8
0: 1+2/3*6-5
1: 1+2-3*4/6
2: 1-2/3*6+5
3: 1-2+3/6*8
4: 1-2*3/6+4
5: 1-2+3/4*8
6: 1-2/3*6+9
7: 1-2*3/6+7
8: 1-2*3/6+8
9: 1-2*3/6+9
10: 1+2*6-9/3
11: 1-2+4/3*9
12: 1+2*7-9/3
13: 1-2+6/3*7
14: 1-2+5/3*9
15: 1-2+6/3*8
16: 1-3+4/2*9
17: 1-2+6/3*9
18: 1-3+5/2*8
19: 1-3+6/2*7
20: 1-2+7/3*9
21: 1-4+6/2*8
22: 1-3+6/2*8
23: 1-2+8/3*9
24: 1+3*9-8/2
25: 1-3+6/2*9
26: 1-3+7/2*8
27: 1+4*7-6/3
28: 1+5*6-9/3
29: 1-4/2+5*6
30: 1+4*8-6/2
31: 1+4*8-6/3
32: 1+5*7-8/2
33: 1-4+8/2*9
34: 1-3+8/2*9
35: 1+4*9-6/3
36: 2/1+5*8-6
37: 3/1-2+4*9
38: 1+5*8-6/2
39: 1-4/2+5*8
40: 1+6*7-9/3
41: 1-4/2+6*7
42: 1+5*9-8/2
43: 1+5*9-6/2
44: 1-4/2+5*9
45: 2/1-5+6*8
46: 1+6*8-9/3
47: 1-4/2+6*8
48: 2/1+6*9-8
49: 2/1+6*9-7

```
50: 3/1+6*9-7
51: 1+6*9-8/2
52: 2/1-4+6*9
53: 1-4/2+6*9
54: 1-6/2+7*8
55: 1-4/2+7*8
56: 2-6/3+7*8
57: 2/1+7*9-8
58: 3/1+7*9-8
59: 2/1-6+7*9
60: 1+7*9-8/2
61: 1-6/2+7*9
62: 1-4/2+7*9
63: 2-6/3+7*9
64: 3/1-2+7*9
65: 4/1-2+7*9
66: 5/1-2+7*9
67: 2/1-7+8*9
68: 2/1-6+8*9
69: 2/1-5+8*9
70: 1-6/2+8*9
71: 1-4/2+8*9
72: 2-6/3+8*9
73: 3/1-2+8*9
74: 4/1-2+8*9
75: 5/1-2+8*9
76: 6/1-2+8*9
77: 7/1-2+8*9
78: None
79: None
```

Número de operaciones del algoritmo de Fuerza Bruta

Queremos obtener la cantidad de operaciones elementales que realiza el algoritmo de fuerza bruta anterior. Suponiendo que lista_numeros tiene tamaño N y que lista_operaciones tiene tamaño M tenemos que:

Primero se realizan 2 asignaciones:

```
nums = lista_numeros
ops = lista_operaciones
```

A continuación se itera sobre una lista de n elementos, con lo cual de momento hay 2+N operaciones elementales:

```
nums = lista_numeros
ops = lista_operaciones

#Iteramos sobre la lista
for num_uno in nums:
```

Después se vuelve a iterar sobre la misma lista, quedando de momento 2+ N*N operaciones elementales:

```
for num_uno in nums:
    for num_dos in nums:
```

A continuación se realiza 1 comprobación y, después se vuelve a iterar sobre la misma lista, quedando 2+N*N(1+N) operaciones elementales:

```
for num_uno in nums:
    for num_dos in nums:
        if num_dos == num_uno:
            continue
        for num_tres in nums:
```

Procediendo de esta forma, tras iterar por cuarta vez sobre la lista de operaciones y comprobar si op_cuatro es op_uno, op_dos u op_tres , tenemos que se realizan 2+N^2(1+N(2+N(3+N(4+M^2(1+M(2+3M)))))) operaciones elementales

```
nums = lista_numeros
ops = lista_operaciones

#Iteramos sobre la lista de números tomando 5 números distintos para comprobar
for num_uno in nums:
    for num_dos in nums:
        if num_dos == num_uno:
            continue
        for num_tres in nums:
            if num_tres == num_uno or num_tres == num_dos:
                continue
            for num_cuatro in nums:
                if num_cuatro == num_uno or num_cuatro == num_dos or num_cuatro == num_tres:
                    continue
                for num_cinco in nums:
                    if (num_cinco == num_uno or num_cinco == num_dos or
                        num_cinco == num_tres or num_cinco == num_cuatro):
                        continue

                    #Iteramos sobre la lista de operaciones tomando 4 distintas para comprobar
                    for op_uno in ops:
                        for op_dos in ops:
                            if op_dos == op_uno:
                                continue
                            for op_tres in ops:
                                if op_tres == op_uno or op_tres == op_dos:
                                    continue
                                for op_cuatro in ops:
                                    if op_cuatro == op_uno or op_cuatro == op_dos or op_cuatro == op_tres:
                                        continue
```

Por último, asignamos un string a la variable expr, realizamos 1 comparación y devolvemos expresión en caso que el resultado de la comparación sea verdadero. Con lo cual tendremos que se realizan 2+N^2(1+N(2+N(3+N(4+M^2(1+M(2+M(3+4))))))). Después se devuelve None si no se encuentra ninguna combinación, siendo el número final de operaciones de 2+N^2(1+N(2+N(3+N(4+M^2(1+M(2+M(3+4))))))) = (7M^4+2M^3+M^2+4)N^5+3N^4+2N^3+N^2+3

```
expr = f"{num_uno}{op_uno}{num_dos}{op_dos}{num_tres}{op_tres}{num_cuatro}{op_cuatro}{num_cinco}"
if eval(expr) == target:
    return expr

return None #Si no encontramos ninguna combinación devolvemos None
```

Cabe notar que hemos supuesto que la asignación de una lista a una variable realizaba tan solo 1 operación. Además, también hemos supuesto que continue no consumía operaciones, que cuando se realizaban varias comprobaciones dentro de un if solo se

realizaba una operación elemental por comprobación y que eval realizaba una sola operación. El número de operaciones exacto puede variar de cambiar estas suposiciones.

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

De la discusión anterior se deduce que el algoritmo tiene 9 bucles for anidados: 5 recorren la lista de números, de tamaño N, y los otros 4 recorren la lista de operaciones. Como el número de operaciones que se puede pasar está acotado, ese factor se considera constante. Además, el resto de instrucciones de la función son de orden constante, por lo que no afectan al orden de crecimiento. En consecuencia, la complejidad temporal del algoritmo es $\mathcal{O}(N^5)$.

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta.

Respuesta

In [ ]:
```
#Se definen dos listas con los números y las operaciones
lista_operaciones=["+","-","*","/"]
lista_numeros=[1,2,3,4,5,6,7,8,9]

def operaciones_para(target):
  #Se crea una copia de las listas anteriores
  temp_ops = lista_operaciones[:]
  temp_nums = lista_numeros[:]

  def buscarDigito(stage, actualExpr):

    #Pequeña poda si ya es imposible llegar al valor cuando queda solo "+" y "-"
    if stage > 0:
      solo_mas_menos = True
      for op in temp_ops: #M operaciones
        if op != "+" and op != "-":#Operaciones constantes
          solo_mas_menos = False
          break
      if solo_mas_menos:
        actualVal = eval(actualExpr)
        s = 0
        for x in temp_nums:#N operaciones
          s += x
        if target < actualVal - s or target > actualVal + s:#Operaciones constan
          return None

    #Si estamos realizando el último paso evaluamos la expresión
    if stage == 5: #Operaciones constantes
      if eval(actualExpr) == target:
```

```
        return actualExpr
      return None

    #Se aplica Backtracking para obtener los números y las operaciones
    for i in range(len(temp_nums)): #N operaciones
      actual_dig = temp_nums.pop(i)
      if stage == 0:
          ans = buscarDigito(stage + 1, str(actual_dig))
      else:
        ans = None
        for j in range(len(temp_ops)): #M operaciones
          actual_op = temp_ops.pop(j) #Operaciones constantes

          ans = buscarDigito(stage + 1, actualExpr + actual_op + str(actual_dig)

          temp_ops.insert(j, actual_op) #Operaciones constantes
          if ans is not None: #Operaciones constantes
            break

      temp_nums.insert(i, actual_dig) #Operaciones constantes
      if ans is not None: #Operaciones constantes
        return ans

    return None

  return buscarDigito(0, "")

for s in range(20):
  print(str(s)+": ",operaciones_para(s))
```

```
0:  1+2/3*6-5
1:  1+2-3*4/6
2:  1+2/3*9-5
3:  1+2/3*9-4
4:  1-2*3/6+4
5:  1+2*3-8/4
6:  1+2*4-9/3
7:  1+2*4-6/3
8:  1+2*5-9/3
9:  1+2*5-6/3
10:  1+2*6-9/3
11:  1+2*6-8/4
12:  1+2*7-9/3
13:  1+2*7-6/3
14:  1+2*8-9/3
15:  1+2*8-6/3
16:  1-3+4/2*9
17:  1+2*9-6/3
18:  1+3*7-8/2
19:  1+3*7-6/2
```

Número de operaciones del algoritmo de Backtracking

Queremos obtener la cantidad de operaciones elementales que realiza el algoritmo de backtracking anterior. Suponemos que lista_numeros tiene tamaño N y que lista_operaciones tiene tamaño M.

Primero se realizan 2 asignaciones:

```
temp_ops = lista_operaciones[:]
temp_nums = lista_numeros[:]
```

Después de las asignaciones se implementa una poda y, a continuación, el backtracking.

Veamos primero las operaciones elementales de la poda:

```
#Pequeña poda si ya es imposible llegar al valor cuando queda solo "+" y "-"
if stage > 0:
  solo_mas_menos = True
  for op in temp_ops: #M operaciones
    if op != "+" and op != "-":#Operaciones constantes
      solo_mas_menos = False
      break
  if solo_mas_menos:
    actualVal = eval(actualExpr)
    s = 0
    for x in temp_nums:#N operaciones
      s += x
    if target < actualVal - s or target > actualVal + s:#Operaciones constantes
      return None
```

Se realiza una comprobación al principio y, después, se realiza una asignación. A continuación, se itera sobre la lista de operaciones, de tamaño M. Dentro de esa iteración se realizan dos comprobaciones más y una asignación, quedando el número de operaciones elementales de la poda en (2+3M) hasta este punto.

```
if stage > 0:
  solo_mas_menos = True
  for op in temp_ops: #M operaciones
    if op != "+" and op != "-":#Operaciones constantes
      solo_mas_menos = False
      break
```

Por otro lado, se comprueba si la variable solo_mas_menos es True, se evalua la variable actualExpr, se asigna el resultado a otra variable y se asigna un valor a la variable s. Después se itera sobre la lista de números y se incrementa el valor de s. Por último se realizan 2 comprobaciones y un return, con lo cual tenemos que el número de operaciones elementales de la poda es de 2+3M+(4+N+3)=3M+N+9

```
if solo_mas_menos:
  actualVal = eval(actualExpr)
  s = 0
  for x in temp_nums:#N operaciones
    s += x
  if target < actualVal - s or target > actualVal + s:#Operaciones constantes
    return None
```

Veamos ahora las operaciones elementales de la implementación del Backtracking:

```
#Si estamos realizando el último paso evaluamos la expresión
if stage == 5: #Operaciones constantes
  if eval(actualExpr) == target:
    return actualExpr
  return None

#Se aplica Backtracking para obtener los números y las operaciones
for i in range(len(temp_nums)): #N operaciones
  actual_dig = temp_nums.pop(i)
  if stage == 0:
      ans = buscarDigito(stage + 1, str(actual_dig))
  else:
    ans = None
    for j in range(len(temp_ops)): #M operaciones
      actual_op = temp_ops.pop(j) #Operaciones constantes

      ans = buscarDigito(stage + 1, actualExpr + actual_op + str(actual_dig))

      temp_ops.insert(j, actual_op) #Operaciones constantes
      if ans is not None: #Operaciones constantes
        break

  temp_nums.insert(i, actual_dig) #Operaciones constantes
  if ans is not None: #Operaciones constantes
    return ans
```

Primero se realiza una comprobación para el caso en que stage sea 5. En esta comprobación se realizan 5 operaciones elementales. Sin embargo, solo entrará en el if en el quinto stage.

```
if stage == 5: #Operaciones constantes
  if eval(actualExpr) == target:
    return actualExpr
  return None
```

Después se itera, se realiza una asignación, se usa la función pop, se realiza una comprobación y dentro del if, otra asignación. En esta se asigna el resultado de ejecutar el propio algoritmo (haciendo 3 operaciones durante la asignación), quedando entonces el número de operaciones hasta ahora en 1+N(4+3*numero_operaciones_buscarDigito) (se suma 1 por la comprobación del if)

```
for i in range(len(temp_nums)): #N operaciones
  actual_dig = temp_nums.pop(i)
  if stage == 0:
      ans = buscarDigito(stage + 1, str(actual_dig))
  else:
```

Por otro lado, dentro del siguiente else se realiza una asignación y, más tarde, se itera. Dentro del bucle, se realizan 2 asignaciones, se usan las funciones pop e insert y se realiza una comprobación. Además, una de las asignaciones llama a la propia función buscar Digito, con lo cual el número de operaciones hasta ahora es 1+N(4+3*numero_operaciones_buscarDigito)+1+M(5+3numero_operaciones_buscarDigito)

```
if stage == 0:
    ans = buscarDigito(stage + 1, str(actual_dig))
else:
  ans = None
  for j in range(len(temp_ops)): #M operaciones
    actual_op = temp_ops.pop(j) #Operaciones constantes

    ans = buscarDigito(stage + 1, actualExpr + actual_op + str(actual_dig)

    temp_ops.insert(j, actual_op) #Operaciones constantes
    if ans is not None: #Operaciones constantes
      break
```

Para finalizar la implementación del Backtracking, se realiza 1 insert, una comprobación y un return. Por tanto, tenemos 1+N(4+3*numero_operaciones_buscarDigito)+1+M(5+3numero_operaciones_buscarDigito)+

```
temp_nums.insert(i, actual_dig) #Operaciones constantes
if ans is not None: #Operaciones constantes
  return ans
```

Por último, para finalizar la función buscarDigito, ser realiza un return. Después se realiza un return dentro de la función operaciones_para con el fin de ejecutar la función buscarDigito desde stage=0.

```
  return None

return buscarDigito(0, "")
```

Por tanto, el número total de operaciones teniendo en cuenta que buscarDigito se ejecuta 5 veces, es 2+(3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+3+1))+3+1) =

$$
\begin{aligned}
&= 20M^5N^5 + 9M^5N^4 + 107M^4N^5 + 69M^4N^4 + 9M^4N^3 + 228M^3N^5 \\
&+ 193M^3N^4 + 60M^3N^3 + 9M^3N^2 + 242M^2N^5 + 255M^2N^4 \\
&+ 133M^2N^3 + 51M^2N^2 + 9M^2N + 128MN^5 + 162MN^4 \\
&+ 122MN^3 + 82MN^2 + 42MN + 3M + 27N^5 + 40N^4 \\
&+ 40N^3 + 40N^2 + 38N + 10
\end{aligned}
$$

(*)Calcula la complejidad del algoritmo.

Respuesta

En este caso, el algoritmo anterior tiene en el peor de los casos un orden de complejidad $\mathcal{O}(N^6)$. Aunque sigue explorando combinaciones de 5 números distintos, la poda introduce un coste adicional que puede obligar a recorrer los números restantes en muchas llamadas, aumentando así la complejidad desde $\mathcal{O}(N^5)$ hasta $\mathcal{O}(N^6)$ en el peor caso. No obstante, en la práctica puede realizar menos operaciones que el algoritmo de fuerza bruta, ya que la poda puede evitar explorar muchas ramas y reducir el tiempo real de ejecución en bastantes casos.

¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?

Respuesta

Podemos establecer unas cotas superior e inferior a la operación y utilizar el algoritmo que hemos diseñado para encontrar el máximo y el mínimo.

Trivialmente, el número *99+9-1/1=90 puede ejercer de cota superior, mientras que 1/1+1-99*=-79 puede ejercer de cota inferior.

Aplicamos el algoritmo:

In [ ]:
```
for i in range(90,0,-1):
  x = operaciones_para(i)
  if x is not None:
    print(x, "=", eval(x))
    break
```

```
7/1-2+8*9 = 77.0
```

In [ ]:
```
for i in range(-79,0):
  x = operaciones_para(i)
  if x is not None:
    print(x, "=", eval(x))
    break
```

```
1+4/2-8*9 = -69.0
```

Por tanto, el máximo es 77 y el mínimo es -69

¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo ?

Respuesta

Ejecutando el siguiente código, podemos comprobar como el algoritmo es capaz de encontrar todos los números entre el -69 y el 77:

In [ ]:
```
comprobacion = []
for s in range(-69,78):
  comprobacion.append(eval(operaciones_para(s)))
if list(range(-69,78)) == comprobacion:
  print("Es capaz.")
else:
  print("No es capaz.")
```

```
Es capaz.
```

Es decir, el algoritmo de FuerzaBruta y el de Backtracking son capaces de calcular todos los números de dicho intervalo, y que llegan al mismo resultado. A continuación añadimos un código donde se ve explícitamente esto:

In [ ]:
```
for s in range(-69,78):
  fb = eval(operaciones_paraFB(s))
  bt = eval(operaciones_para(s))
  if s == bt and fb == bt:
    print(operaciones_paraFB(s), " = ", s , " = ",operaciones_para(s))
  elif s != fb:
    print("El algoritmo de Fuerza Bruta no alcanza ", s)
  elif s != bt:
    print("El algoritmo de Backtracking no alcanza ", s)
  else:
    print("Error")
```

1+4/2-8*9	=	-69	=	1+4/2-8*9
1+6/2-8*9	=	-68	=	1+6/2-8*9
2/1+3-8*9	=	-67	=	2/1+3-8*9
2/1+4-8*9	=	-66	=	2/1+4-8*9
2/1+5-8*9	=	-65	=	2/1+5-8*9
2/1+6-8*9	=	-64	=	2/1+6-8*9
2/1+7-8*9	=	-63	=	2/1+7-8*9
3/1+7-8*9	=	-62	=	3/1+7-8*9
4/1+7-8*9	=	-61	=	4/1+7-8*9
1+4/2-7*9	=	-60	=	1+4/2-7*9
1+6/2-7*9	=	-59	=	1+6/2-7*9
1-7*9+8/2	=	-58	=	1-7*9+8/2
2/1+4-7*9	=	-57	=	2/1+4-7*9
2/1+5-7*9	=	-56	=	2/1+5-7*9
2/1+6-7*9	=	-55	=	2/1+6-7*9
3/1+6-7*9	=	-54	=	3/1+6-7*9
1+4/2-7*8	=	-53	=	1+4/2-7*8
1+6/2-7*8	=	-52	=	1+6/2-7*8
1+4/2-6*9	=	-51	=	1+4/2-6*9
2/1+4-7*8	=	-50	=	2/1+4-7*8
1-6*9+8/2	=	-49	=	1-6*9+8/2
2/1+4-6*9	=	-48	=	2/1+4-6*9
2/1+5-6*9	=	-47	=	2/1+5-6*9
3/1+5-6*9	=	-46	=	3/1+5-6*9
1+4/2-6*8	=	-45	=	1+4/2-6*8
1-6*8+9/3	=	-44	=	1-6*8+9/3
2/1+3-6*8	=	-43	=	2/1+3-6*8
1+4/2-5*9	=	-42	=	1+4/2-5*9
1-5*9+6/2	=	-41	=	1-5*9+6/2
1-5*9+8/2	=	-40	=	1-5*9+8/2
1+4/2-6*7	=	-39	=	1+4/2-6*7
1-6*7+9/3	=	-38	=	1-6*7+9/3
1+4/2-5*8	=	-37	=	1+4/2-5*8
1-5*8+6/2	=	-36	=	1-5*8+6/2
2/1+3-5*8	=	-35	=	2/1+3-5*8
2/1+4-5*8	=	-34	=	2/1+4-5*8
1-4*9+6/3	=	-33	=	1-4*9+6/3
1+3-8/2*9	=	-32	=	1+3-8/2*9
1+4-8/2*9	=	-31	=	1+4-8/2*9
1-5*7+8/2	=	-30	=	1+5-8/2*9
1-4*8+6/3	=	-29	=	1-4*8+6/3
1-4*8+6/2	=	-28	=	1-4*8+6/2
1+4/2-5*6	=	-27	=	1+4/2-5*6
1-5*6+9/3	=	-26	=	1-5*6+9/3
1-4*7+6/3	=	-25	=	1-4*7+6/3
1+3-7/2*8	=	-24	=	1+3-7/2*8
1+3-6/2*9	=	-23	=	1+3-6/2*9
1-3*9+8/2	=	-22	=	1-3*9+8/2
1+2-8/3*9	=	-21	=	1+2-8/3*9
1+3-6/2*8	=	-20	=	1+3-6/2*8
1+4-6/2*8	=	-19	=	1+4-6/2*8
1+2-7/3*9	=	-18	=	1+2-7/3*9
1+3-6/2*7	=	-17	=	1+3-6/2*7
1+3-5/2*8	=	-16	=	1+3-5/2*8
1+2-6/3*9	=	-15	=	1+2-6/3*9
1+3-4/2*9	=	-14	=	1+3-4/2*9
1+2-6/3*8	=	-13	=	1+2-6/3*8
1+2-5/3*9	=	-12	=	1+2-5/3*9
1+2-6/3*7	=	-11	=	1+2-6/3*7
1-2*7+9/3	=	-10	=	1-2*7+9/3

1+2-4/3*9	=	-9	=	1+2-4/3*9
1-2*6+9/3	=	-8	=	1-2*6+9/3
1+2*3/6-9	=	-7	=	1+2*3/6-9
1+2*3/6-8	=	-6	=	1+2*3/6-8
1+2*3/6-7	=	-5	=	1+2*3/6-7
1+2/3*6-9	=	-4	=	1+2/3*6-9
1+2-3/4*8	=	-3	=	1+2-3/4*8
1+2*3/6-4	=	-2	=	1+2*3/6-4
1+2-3/6*8	=	-1	=	1+2-3/6*8
1+2/3*6-5	=	0	=	1+2/3*6-5
1+2-3*4/6	=	1	=	1+2-3*4/6
1-2/3*6+5	=	2	=	1+2/3*9-5
1-2+3/6*8	=	3	=	1+2/3*9-4
1-2*3/6+4	=	4	=	1-2*3/6+4
1-2+3/4*8	=	5	=	1+2*3-8/4
1-2/3*6+9	=	6	=	1+2*4-9/3
1-2*3/6+7	=	7	=	1+2*4-6/3
1-2*3/6+8	=	8	=	1+2*5-9/3
1-2*3/6+9	=	9	=	1+2*5-6/3
1+2*6-9/3	=	10	=	1+2*6-9/3
1-2+4/3*9	=	11	=	1+2*6-8/4
1+2*7-9/3	=	12	=	1+2*7-9/3
1-2+6/3*7	=	13	=	1+2*7-6/3
1-2+5/3*9	=	14	=	1+2*8-9/3
1-2+6/3*8	=	15	=	1+2*8-6/3
1-3+4/2*9	=	16	=	1-3+4/2*9
1-2+6/3*9	=	17	=	1+2*9-6/3
1-3+5/2*8	=	18	=	1+3*7-8/2
1-3+6/2*7	=	19	=	1+3*7-6/2
1-2+7/3*9	=	20	=	1-2+7/3*9
1-4+6/2*8	=	21	=	1+4*6-8/2
1-3+6/2*8	=	22	=	1+3*8-6/2
1-2+8/3*9	=	23	=	1-2+8/3*9
1+3*9-8/2	=	24	=	1+3*9-8/2
1-3+6/2*9	=	25	=	1+3*9-6/2
1-3+7/2*8	=	26	=	1+3*9-4/2
1+4*7-6/3	=	27	=	1+4*7-6/3
1+5*6-9/3	=	28	=	1+5*6-9/3
1-4/2+5*6	=	29	=	1-4/2+5*6
1+4*8-6/2	=	30	=	1+4*8-6/2
1+4*8-6/3	=	31	=	1+4*8-6/3
1+5*7-8/2	=	32	=	1+5*7-8/2
1-4+8/2*9	=	33	=	1+4*9-8/2
1-3+8/2*9	=	34	=	1-3+8/2*9
1+4*9-6/3	=	35	=	1+4*9-6/3
2/1+5*8-6	=	36	=	2/1+5*8-6
3/1-2+4*9	=	37	=	3/1-2+4*9
1+5*8-6/2	=	38	=	1+5*8-6/2
1-4/2+5*8	=	39	=	1-4/2+5*8
1+6*7-9/3	=	40	=	1+6*7-9/3
1-4/2+6*7	=	41	=	1-4/2+6*7
1+5*9-8/2	=	42	=	1+5*9-8/2
1+5*9-6/2	=	43	=	1+5*9-6/2
1-4/2+5*9	=	44	=	1-4/2+5*9
2/1-5+6*8	=	45	=	2/1-5+6*8
1+6*8-9/3	=	46	=	1+6*8-9/3
1-4/2+6*8	=	47	=	1-4/2+6*8
2/1+6*9-8	=	48	=	2/1+6*9-8
2/1+6*9-7	=	49	=	2/1+6*9-7
3/1+6*9-7	=	50	=	3/1+6*9-7

```
1+6*9-8/2  =  51  =  1+6*9-8/2
2/1-4+6*9  =  52  =  2/1-4+6*9
1-4/2+6*9  =  53  =  1-4/2+6*9
1-6/2+7*8  =  54  =  1-6/2+7*8
1-4/2+7*8  =  55  =  1-4/2+7*8
2-6/3+7*8  =  56  =  2-6/3+7*8
2/1+7*9-8  =  57  =  2/1+7*9-8
3/1+7*9-8  =  58  =  3/1+7*9-8
2/1-6+7*9  =  59  =  2/1-6+7*9
1+7*9-8/2  =  60  =  1+7*9-8/2
1-6/2+7*9  =  61  =  1-6/2+7*9
1-4/2+7*9  =  62  =  1-4/2+7*9
2-6/3+7*9  =  63  =  2-6/3+7*9
3/1-2+7*9  =  64  =  3/1-2+7*9
4/1-2+7*9  =  65  =  4/1-2+7*9
5/1-2+7*9  =  66  =  5/1-2+7*9
2/1-7+8*9  =  67  =  2/1-7+8*9
2/1-6+8*9  =  68  =  2/1-6+8*9
2/1-5+8*9  =  69  =  2/1-5+8*9
1-6/2+8*9  =  70  =  1-6/2+8*9
1-4/2+8*9  =  71  =  1-4/2+8*9
2-6/3+8*9  =  72  =  2-6/3+8*9
3/1-2+8*9  =  73  =  3/1-2+8*9
4/1-2+8*9  =  74  =  4/1-2+8*9
5/1-2+8*9  =  75  =  5/1-2+8*9
6/1-2+8*9  =  76  =  6/1-2+8*9
7/1-2+8*9  =  77  =  7/1-2+8*9
```

Argumenta porque crees que el algoritmo anterior mejora el algoritmo por fuerza bruta

Pese a que la heurística empeora, aparentemente, el orden de complejidad del algoritmo, pasando de $\mathcal{O}(N^5)$ a $\mathcal{O}(N^6)$ el algoritmo de Backtracking, el cual tiene mayor coste asintótico que el de Fuerza Bruta, que tiene $\mathcal{O}(N^5)$; en la práctica el algoritmo de Backtracking será más eficiente que el de Fuerza Bruta. Esto es debido a que la poda es capaz de evitar que se recorra todo el árbol en la mitad de los casos (ya que de las 24 combinaciones posibles de operaciones, la mitad acaban en suma o resta), siendo una cuarta parte de los casos totales especialmente eficientes (6 combinaciones en total) dado que acaban en suma y resta (o viceversa).

Esto mismo podemos comprobarlo si hacemos un estudio empírico de nuestro problema. Veamos primero que el algoritmo de Backtracking más rápido que el de Fuerza Bruta en una mayor cantidad de casos:

In [ ]:
```
import time

lista_numeros=[1,2,3,4,5,6,7,8,9]
mejorFB = []
mejorBT = []
igualDeRapidos = []

for s in range(-69,78):
```

```
  startFB = time.perf_counter()
  operaciones_paraFB(s)
  stopFB = time.perf_counter()
  tiempoFB= stopFB-startFB

  startBT = time.perf_counter()
  operaciones_para(s)
  stopBT = time.perf_counter()
  tiempoBT= stopBT-startBT

  if tiempoFB > tiempoBT:
    mejorBT.append(s)
  elif tiempoFB < tiempoBT:
    mejorFB.append(s)
  else:
    igualDeRapidos.append(s)

print("Hay ", len(mejorFB)," números para los cuales el algoritmo de Fuerza Brut
print("Hay ", len(mejorBT)," números para los cuales el algoritmo de Backtrackin
print("Hay ", len(igualDeRapidos)," números para los cuales los algoritmo de Bac
```

```
Hay  49  números para los cuales el algoritmo de Fuerza Bruta es más rápido, y so
n los siguientes:  [-61, -43, -35, -30, -28, -27, -26, -25, -24, -23, -22, -20, -
19, -17, -16, -14, -10, -8, -4, 0, 2, 3, 4, 5, 6, 7, 8, 9, 11, 14, 16, 18, 19, 2
0, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 32, 33, 44, 50, 74]
Hay  98  números para los cuales el algoritmo de Backtracking es más rápido, y so
n los siguientes:  [-69, -68, -67, -66, -65, -64, -63, -62, -60, -59, -58, -57, -
56, -55, -54, -53, -52, -51, -50, -49, -48, -47, -46, -45, -44, -42, -41, -40, -3
9, -38, -37, -36, -34, -33, -32, -31, -29, -21, -18, -15, -13, -12, -11, -9, -7,
-6, -5, -3, -2, -1, 1, 10, 12, 13, 15, 17, 31, 34, 35, 36, 37, 38, 39, 40, 41, 4
2, 43, 45, 46, 47, 48, 49, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 6
4, 65, 66, 67, 68, 69, 70, 71, 72, 73, 75, 76, 77]
Hay  0  números para los cuales los algoritmo de Backtracking y Fuerza Bruta son
igual de rápidos, y son los siguientes:  []
```

Cabría preguntarse también cuál es el tiempo medio que tarda el algoritmo de Backtracking en obtener el número que busca, y si es más rápido en promedio que el algoritmo de Fuerza Bruta. Veamos esto empíricamente:

In [ ]:

```
timeFB=[]
timeBT=[]
for s in range(-69,78):
  startFB = time.perf_counter()
  operaciones_paraFB(s)
  stopFB = time.perf_counter()
  tiempoFB= stopFB-startFB
  timeFB.append(tiempoFB)

  startBT = time.perf_counter()
  operaciones_para(s)
  stopBT = time.perf_counter()
  tiempoBT= stopBT-startBT
  timeBT.append(tiempoBT)

tiempoMedioFB = sum(timeFB)/len(timeFB)
tiempoMedioBT = sum(timeBT)/len(timeBT)

if tiempoMedioFB > tiempoMedioBT:
  print(f"El algoritmo de Backtracking con {tiempoMedioBT}s es más rápido en pro
```

```
elif tiempoMedioFB < tiempoMedioBT:
  print(f"El algoritmo de Fuerza Bruta con {tiempoMedioFB}s es más rápido en pro
else:
  print("Son igual de rápidos.")
print(f"Diferencia en el tiempo promedio: {tiempoMedioFB/tiempoMedioBT}")
```

```
El algoritmo de Backtracking con 0.2560937790137551s es más rápido en promedio qu
e el de Fuerza Bruta con 0.3253234458029665s.
Diferencia en el tiempo promedio: 1.2703293576900712
```

Por tanto, hemos visto que el algoritmo de Backtracking es, efectivamente, más rápido que el de Fuerza Bruta tanto en promedio como en cantidad de casos. Cabría entonces preguntarse si existe alguna otra versión del problema para la cuál es más eficiente el algoritmo de Fuerza Bruta que el de BackTracking.

Vemeros esto empíricamente, reduciendo gradualmente la cantidad de números hasta llegar al resultado. Hay que tener en cuenta que la cantidado de números, debido al diseño de los algoritmos, no puede ser menor que 5. De igual manera, el numero de operaciones no puede ser menor que 4. Además ajustaremos el rango para que calcule todos los números que sea capaz.

8 números

Creamos una función que calcule los números que son capaces de calcular los algoritmos:

In [ ]:
```
def rangoAlgs():
  lista_comprobacion=[]

  #Iteramos sobre el rango que es capaz de sacar el problemaoriginal
  for s in range(-69,78):
    fb0 = operaciones_paraFB(s)
    bt0 = operaciones_para(s)

    #Si ambos algoritmos son capaces de calcular el número, lo almacenamos
    if fb0 is not None and bt0 is not None:
      fb = eval(fb0)
      bt = eval(bt0)
      if s == bt and fb == bt:
        print(operaciones_paraFB(s), " = ", s , " = ",operaciones_para(s))
        lista_comprobacion.append(s)
      else:
        return "Error1"
    elif (fb0 is None and bt0 is not None) or (fb0 is not None and bt0 is None):
      return "Error2"
  return lista_comprobacion
```

Creamos una función que comprueba los tiempos:

In [ ]:
```
def comprobacionTiempos():

  #Declaramos la variables que vamos a usar y nos traemos la lista de números qu
  lista_comprobacion=rangoAlgs()
  mejorFB = []
  mejorBT = []
  igualDeRapidos = []
```

```
timeFB=[]
timeBT=[]

#Recorremos lista_comprobación y vamos obteniendo los tiempos y la lista de ti
for s in lista_comprobacion:
  startFB = time.perf_counter()
  operaciones_paraFB(s)
  stopFB = time.perf_counter()
  tiempoFB= stopFB-startFB
  timeFB.append(tiempoFB)

  startBT = time.perf_counter()
  operaciones_para(s)
  stopBT = time.perf_counter()
  tiempoBT= stopBT-startBT
  timeBT.append(tiempoBT)

  #Obtenemos las listas de los números con los que cada algortimo es más rápid
  if tiempoFB > tiempoBT:
    mejorBT.append(s)
  elif tiempoFB < tiempoBT:
    mejorFB.append(s)
  else:
    igualDeRapidos.append(s)

#Obtenemos los tiempos medios y mostramos los números con los que cada algorit
tiempoMedioFB = sum(timeFB)/len(timeFB)
tiempoMedioBT = sum(timeBT)/len(timeBT)
print("Hay ", len(mejorFB)," números para los cuales el algoritmo de Fuerza Br
print("Hay ", len(mejorBT)," números para los cuales el algoritmo de Backtrack
print("Hay ", len(igualDeRapidos)," números para los cuales los algoritmo de B

if tiempoMedioFB > tiempoMedioBT:
  print(f"El algoritmo de Backtracking con {tiempoMedioBT}s es más rápido en p
elif tiempoMedioFB < tiempoMedioBT:
  print(f"El algoritmo de Fuerza Bruta con {tiempoMedioFB}s es más rápido en p
else:
  print("Son igual de rápidos.")
print(f"Diferencia en el tiempo promedio: {tiempoMedioFB/tiempoMedioBT}")

return None
```

Realizamos las comprobaciones:

In [ ]:
```
lista_numeros=[1,2,3,4,5,6,7,8]
comprobacionTiempos()
```

```
1+4/2-7*8  =  -53  =  1+4/2-7*8
1+6/2-7*8  =  -52  =  1+6/2-7*8
2/1+3-7*8  =  -51  =  2/1+3-7*8
2/1+4-7*8  =  -50  =  2/1+4-7*8
2/1+5-7*8  =  -49  =  2/1+5-7*8
2/1+6-7*8  =  -48  =  2/1+6-7*8
3/1+6-7*8  =  -47  =  3/1+6-7*8
4/1+6-7*8  =  -46  =  4/1+6-7*8
1+4/2-6*8  =  -45  =  1+4/2-6*8
2/1+3-6*8  =  -43  =  2/1+3-6*8
2/1+4-6*8  =  -42  =  2/1+4-6*8
2/1+5-6*8  =  -41  =  2/1+5-6*8
3/1+5-6*8  =  -40  =  3/1+5-6*8
1+4/2-6*7  =  -39  =  1+4/2-6*7
2-6*7+8/4  =  -38  =  2-6*7+8/4
1+4/2-5*8  =  -37  =  1+4/2-5*8
1-5*8+6/2  =  -36  =  1-5*8+6/2
2/1+3-5*8  =  -35  =  2/1+3-5*8
2/1+4-5*8  =  -34  =  2/1+4-5*8
3/1+4-5*8  =  -33  =  3/1+4-5*8
1+4/2-5*7  =  -32  =  1+4/2-5*7
1-5*7+6/2  =  -31  =  1-5*7+6/2
1-5*7+8/2  =  -30  =  1-5*7+8/2
1-4*8+6/3  =  -29  =  1-4*8+6/3
1-4*8+6/2  =  -28  =  1-4*8+6/2
1+4/2-5*6  =  -27  =  1+4/2-5*6
2-5*6+8/4  =  -26  =  2-5*6+8/4
1-4*7+6/3  =  -25  =  1-4*7+6/3
1+3-7/2*8  =  -24  =  1+3-7/2*8
1+4-7/2*8  =  -23  =  1+4-7/2*8
1+5-7/2*8  =  -22  =  1+5-7/2*8
1-3*8+4/2  =  -21  =  1-3*8+4/2
1+3-6/2*8  =  -20  =  1+3-6/2*8
1+4-6/2*8  =  -19  =  1+4-6/2*8
1-3*7+4/2  =  -18  =  1-3*7+4/2
1+3-6/2*7  =  -17  =  1+3-6/2*7
1+3-5/2*8  =  -16  =  1+3-5/2*8
1-3*6+4/2  =  -15  =  1-3*6+4/2
2/1-3*7+5  =  -14  =  2/1-3*7+5
1+2-6/3*8  =  -13  =  1+2-6/3*8
1+3-4/2*8  =  -12  =  1+3-4/2*8
1+2-6/3*7  =  -11  =  1+2-6/3*7
1+3-4/2*7  =  -10  =  1+3-4/2*7
1+2-6/4*8  =  -9  =  1+2-6/4*8
1+3-4/2*6  =  -8  =  1+3-4/2*6
1+2-5/3*6  =  -7  =  1+2-5/3*6
1+2*3/6-8  =  -6  =  1+2*3/6-8
1+2*3/6-7  =  -5  =  1+2*3/6-7
1+2/4*6-8  =  -4  =  1+2*4/8-6
1+2-3/4*8  =  -3  =  1+2-3/4*8
1+2*3/6-4  =  -2  =  1+2*3/6-4
1+2-3/6*8  =  -1  =  1+2-3/6*8
1+2/3*6-5  =  0  =  1+2/3*6-5
1+2-3*4/6  =  1  =  1+2-3*4/6
1-2/3*6+5  =  2  =  1+2/4*8-3
1-2+3/6*8  =  3  =  1-2+3/6*8
1-2*3/6+4  =  4  =  1-2*3/6+4
1-2+3/4*8  =  5  =  1+2*3-8/4
1/2*4-3+7  =  6  =  1-2*4/8+6
1-2*3/6+7  =  7  =  1+2*4-6/3
```

```
1-2*3/6+8  =  8  =  1-2*3/6+8
1-2+5/3*6  =  9  =  1+2*5-6/3
1-3+4/2*6  =  10  =  1+3*4-6/2
1-2+6/4*8  =  11  =  1+2*6-8/4
1-3+4/2*7  =  12  =  1+3*5-8/2
1-2+6/3*7  =  13  =  1+2*7-6/3
1-3+4/2*8  =  14  =  1+3*5-4/2
1-2+6/3*8  =  15  =  1+2*8-6/3
2/1+3*6-4  =  16  =  2/1+3*6-4
1+3*6-4/2  =  17  =  1+3*6-4/2
1-3+5/2*8  =  18  =  1+3*7-8/2
1-3+6/2*7  =  19  =  1+3*7-6/2
1+3*7-4/2  =  20  =  1+3*7-4/2
1-4+6/2*8  =  21  =  1+4*6-8/2
1-3+6/2*8  =  22  =  1+3*8-6/2
1+3*8-4/2  =  23  =  1+3*8-4/2
1-5+7/2*8  =  24  =  1-5+7/2*8
1-4+7/2*8  =  25  =  1+4*7-8/2
1-3+7/2*8  =  26  =  1-3+7/2*8
1+4*7-6/3  =  27  =  1+4*7-6/3
2/1-4+5*6  =  28  =  2/1+4*8-6
1-4/2+5*6  =  29  =  1-4/2+5*6
1+4*8-6/2  =  30  =  1+4*8-6/2
1+4*8-6/3  =  31  =  1+4*8-6/3
1+5*7-8/2  =  32  =  1+5*7-8/2
1+5*7-6/2  =  33  =  1+5*7-6/2
1-4/2+5*7  =  34  =  1-4/2+5*7
2/1+5*8-7  =  35  =  2/1+5*8-7
2/1+5*8-6  =  36  =  2/1+5*8-6
3/1+5*8-6  =  37  =  3/1+5*8-6
1+5*8-6/2  =  38  =  1+5*8-6/2
1-4/2+5*8  =  39  =  1-4/2+5*8
2/1-4+6*7  =  40  =  2/1-4+6*7
1-4/2+6*7  =  41  =  1-4/2+6*7
2+6*7-8/4  =  42  =  2+6*7-8/4
2/1+6*8-7  =  43  =  2/1+6*8-7
3/1+6*8-7  =  44  =  3/1+6*8-7
2/1-5+6*8  =  45  =  2/1-5+6*8
2/1-4+6*8  =  46  =  2/1-4+6*8
1-4/2+6*8  =  47  =  1-4/2+6*8
6/1*7-2+8  =  48  =  6/1*7-2+8
3/1-2+6*8  =  49  =  3/1-2+6*8
4/1-2+6*8  =  50  =  4/1-2+6*8
5/1-2+6*8  =  51  =  5/1-2+6*8
2/1-6+7*8  =  52  =  2/1-6+7*8
2/1-5+7*8  =  53  =  2/1-5+7*8
1-6/2+7*8  =  54  =  1-6/2+7*8
1-4/2+7*8  =  55  =  1-4/2+7*8
2-6/3+7*8  =  56  =  2-6/3+7*8
3/1-2+7*8  =  57  =  3/1-2+7*8
4/1-2+7*8  =  58  =  4/1-2+7*8
5/1-2+7*8  =  59  =  5/1-2+7*8
6/1-2+7*8  =  60  =  6/1-2+7*8
Hay  31  números para los cuales el algoritmo de Fuerza Bruta es más rápido, y so
n los siguientes:  [-19, -18, -17, -16, -15, -14, -12, -10, -8, 2, 3, 4, 5, 6, 7,
8, 10, 12, 14, 16, 17, 18, 19, 20, 21, 22, 23, 24, 28, 29, 42]
Hay  82  números para los cuales el algoritmo de Backtracking es más rápido, y so
n los siguientes:  [-53, -52, -51, -50, -49, -48, -47, -46, -45, -43, -42, -41, -
40, -39, -38, -37, -36, -35, -34, -33, -32, -31, -30, -29, -28, -27, -26, -25, -2
4, -23, -22, -21, -20, -13, -11, -9, -7, -6, -5, -4, -3, -2, -1, 0, 1, 9, 11, 13,
```

```
15, 25, 26, 27, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 43, 44, 45, 46, 4
7, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60]
Hay  0  números para los cuales los algoritmo de Backtracking y Fuerza Bruta son
igual de rápidos, y son los siguientes:  []
El algoritmo de Backtracking con 0.1486189211502391s es más rápido en promedio qu
e el de Fuerza Bruta con 0.18336195869901087s.
Diferencia en el tiempo promedio: 1.2337726399833704
```

Por tanto, el algoritmo de Backtracking continua siendo más rápido en promedio y, además, también es más rápido en la mayoría de los casos.

7 números

In [ ]:
```
lista_numeros=[1,2,3,4,5,6,7]
comprobacionTiempos()
```

```
1+4/2-6*7  =  -39  =  1+4/2-6*7
2/1+3-6*7  =  -37  =  2/1+3-6*7
2/1+4-6*7  =  -36  =  2/1+4-6*7
2/1+5-6*7  =  -35  =  2/1+5-6*7
3/1+5-6*7  =  -34  =  3/1+5-6*7
4/1+5-6*7  =  -33  =  4/1+5-6*7
1+4/2-5*7  =  -32  =  1+4/2-5*7
1-5*7+6/2  =  -31  =  1-5*7+6/2
2/1+3-5*7  =  -30  =  2/1+3-5*7
2/1+4-5*7  =  -29  =  2/1+4-5*7
3/1+4-5*7  =  -28  =  3/1+4-5*7
1+4/2-5*6  =  -27  =  1+4/2-5*6
3/1-5*7+6  =  -26  =  3/1-5*7+6
1-4*7+6/3  =  -25  =  1-4*7+6/3
1-4*7+6/2  =  -24  =  1-4*7+6/2
2/1+3-4*7  =  -23  =  2/1+3-4*7
3-4*7+6/2  =  -22  =  3-4*7+6/2
2/1-4*7+5  =  -21  =  2/1-4*7+5
2/1-4*7+6  =  -20  =  2/1-4*7+6
2/1+3-4*6  =  -19  =  2/1+3-4*6
1-3*7+4/2  =  -18  =  1-3*7+4/2
1+3-6/2*7  =  -17  =  1+3-6/2*7
1-4*5+6/2  =  -16  =  1+4-6/2*7
1-3*6+4/2  =  -15  =  1-3*6+4/2
2/1-3*7+5  =  -14  =  2/1-3*7+5
2/1-3*7+6  =  -13  =  2/1-3*7+6
1-3*5+4/2  =  -12  =  1-3*5+4/2
1+2-6/3*7  =  -11  =  1+2-6/3*7
1+3-4/2*7  =  -10  =  1+3-4/2*7
1+4-6/3*7  =  -9  =  1+4-6/3*7
1+3-4/2*6  =  -8  =  1+3-4/2*6
1+2-5/3*6  =  -7  =  1+2-5/3*6
1+3-4/2*5  =  -6  =  1+3-4/2*5
1+2*3/6-7  =  -5  =  1+2*3/6-7
1-3/2*6+4  =  -4  =  1+3*4/6-7
1+2*3/6-5  =  -3  =  1+2*3/6-5
1+2*3/6-4  =  -2  =  1+2*3/6-4
1/2*4+3-6  =  -1  =  1+2/4*6-5
1+2/3*6-5  =  0  =  1+2/3*6-5
1+2-3*4/6  =  1  =  1+2-3*4/6
1-2/3*6+5  =  2  =  1-2/3*6+5
1/2*4-5+6  =  3  =  1-2/4*6+5
1-2*3/6+4  =  4  =  1-2*3/6+4
1-2*3/6+5  =  5  =  1-2*3/6+5
1/2*4-3+7  =  6  =  1/2*4-3+7
1-2*3/6+7  =  7  =  1+2*4-6/3
1-3+4/2*5  =  8  =  1-3+4/2*5
1-2+5/3*6  =  9  =  1+2*5-6/3
1-3+4/2*6  =  10  =  1+3*4-6/2
1-4+6/3*7  =  11  =  1-4+6/3*7
1-3+4/2*7  =  12  =  1-3+4/2*7
1-2+6/3*7  =  13  =  1+2*7-6/3
1+3*5-4/2  =  14  =  1+3*5-4/2
2/1+3*6-5  =  15  =  2-1+6/3*7
2/1+3*6-4  =  16  =  2/1+3*6-4
1+3*6-4/2  =  17  =  1+3*6-4/2
1+4*5-6/2  =  18  =  1+4*5-6/2
1-3+6/2*7  =  19  =  1+3*7-6/2
1+3*7-4/2  =  20  =  1+3*7-4/2
2/1+4*6-5  =  21  =  2/1+4*6-5
```

```
3/1+4*6-5  =  22  =  3/1+4*6-5
2/1-3+4*6  =  23  =  2/1-3+4*6
2/1+4*7-6  =  24  =  2/1+4*7-6
2/1+4*7-5  =  25  =  2/1+4*7-5
1+4*7-6/2  =  26  =  1+4*7-6/2
1+4*7-6/3  =  27  =  1+4*7-6/3
2/1-4+5*6  =  28  =  2/1-4+5*6
1-4/2+5*6  =  29  =  1-4/2+5*6
4/1*7-3+5  =  30  =  4/1*7-3+5
2/1+5*7-6  =  31  =  2/1+5*7-6
3/1+5*7-6  =  32  =  3/1+5*7-6
1+5*7-6/2  =  33  =  1+5*7-6/2
1-4/2+5*7  =  34  =  1-4/2+5*7
2+5*7-6/3  =  35  =  2+5*7-6/3
3/1-2+5*7  =  36  =  3/1-2+5*7
4/1-2+5*7  =  37  =  4/1-2+5*7
5/1*7-3+6  =  38  =  5/1*7-3+6
2/1-5+6*7  =  39  =  2/1-5+6*7
2/1-4+6*7  =  40  =  2/1-4+6*7
1-4/2+6*7  =  41  =  1-4/2+6*7
3/1-2+6*7  =  43  =  3/1-2+6*7
4/1-2+6*7  =  44  =  4/1-2+6*7
5/1-2+6*7  =  45  =  5/1-2+6*7
Hay  24  números para los cuales el algoritmo de Fuerza Bruta es más rápido, y so
n los siguientes:  [-19, -14, -13, -12, -10, -9, -8, -6, -4, 2, 3, 4, 5, 6, 7, 8,
10, 11, 12, 14, 15, 16, 18, 22]
Hay  59  números para los cuales el algoritmo de Backtracking es más rápido, y so
n los siguientes:  [-39, -37, -36, -35, -34, -33, -32, -31, -30, -29, -28, -27, -
26, -25, -24, -23, -22, -21, -20, -18, -17, -16, -15, -11, -7, -5, -3, -2, -1, 0,
1, 9, 13, 17, 19, 20, 21, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36,
37, 38, 39, 40, 41, 43, 44, 45]
Hay  0  números para los cuales los algoritmo de Backtracking y Fuerza Bruta son
igual de rápidos, y son los siguientes:  []
El algoritmo de Backtracking con 0.07949476218087205s es más rápido en promedio q
ue el de Fuerza Bruta con 0.08738108024136823s.
Diferencia en el tiempo promedio: 1.0992055054212588
```

Por tanto, el algoritmo de Backtracking continua siendo más rápido en promedio y, además, también es más rápido en la mayoría de los casos.

6 números

In [ ]:
```
lista_numeros=[1,2,3,4,5,6]
comprobacionTiempos()
```

```
1+4/2-5*6  =  -27  =  1+4/2-5*6
2/1+3-5*6  =  -25  =  2/1+3-5*6
2/1+4-5*6  =  -24  =  2/1+4-5*6
3/1+4-5*6  =  -23  =  3/1+4-5*6
2/1+3-4*6  =  -19  =  2/1+3-4*6
1-4*5+6/3  =  -17  =  1-4*5+6/3
1-4*5+6/2  =  -16  =  1-4*5+6/2
1-3*6+4/2  =  -15  =  1-3*6+4/2
3-4*5+6/2  =  -14  =  3-4*5+6/2
1-3*5+4/2  =  -12  =  1-3*5+4/2
1+3-5/2*6  =  -11  =  1+3-5/2*6
1+4-5/2*6  =  -10  =  1+4-5/2*6
2/1-3*5+4  =  -9  =  2/1-3*5+4
1+3-4/2*6  =  -8  =  1+3-4/2*6
1+2-5/3*6  =  -7  =  1+2-5/3*6
1+3-4/2*5  =  -6  =  1+3-4/2*5
1+2-4/3*6  =  -5  =  1+2-4/3*6
1-3/2*6+4  =  -4  =  1-3/2*6+4
1+2*3/6-5  =  -3  =  1+2*3/6-5
1+2*3/6-4  =  -2  =  1+2*3/6-4
1/2*4+3-6  =  -1  =  1+2/4*6-5
1+2/3*6-5  =  0  =  1+2/3*6-5
1+2-3*4/6  =  1  =  1+2-3*4/6
1-2/3*6+5  =  2  =  1-2/3*6+5
1/2*4-5+6  =  3  =  1-2/4*6+5
1-2*3/6+4  =  4  =  1-2*3/6+4
1-2*3/6+5  =  5  =  1-2*3/6+5
1+3/2*6-4  =  6  =  1+3/2*6-4
1-2+4/3*6  =  7  =  1+2*4-6/3
1-3+4/2*5  =  8  =  1-3+4/2*5
1-2+5/3*6  =  9  =  1+2*5-6/3
1-3+4/2*6  =  10  =  1+3*4-6/2
2/1+3*5-6  =  11  =  2-1+5/3*6
1-4+5/2*6  =  12  =  1-4+5/2*6
1-3+5/2*6  =  13  =  1+3*5-6/2
1+3*5-4/2  =  14  =  1+3*5-4/2
2/1+3*6-5  =  15  =  2/1+3*6-5
2/1+3*6-4  =  16  =  2/1+3*6-4
1+3*6-4/2  =  17  =  1+3*6-4/2
1+4*5-6/2  =  18  =  1+4*5-6/2
1+4*5-6/3  =  19  =  1+4*5-6/3
2+4*5-6/3  =  20  =  2+4*5-6/3
2/1+4*6-5  =  21  =  2/1+4*6-5
3/1+4*6-5  =  22  =  3/1+4*6-5
2/1-3+4*6  =  23  =  2/1-3+4*6
4/1*5-2+6  =  24  =  4/1*5-2+6
3/1-2+4*6  =  25  =  3/1-2+4*6
4/1*6-3+5  =  26  =  4/1*6-3+5
4/1*6-2+5  =  27  =  4/1*6-2+5
2/1-4+5*6  =  28  =  2/1-4+5*6
1-4/2+5*6  =  29  =  1-4/2+5*6
3/1-2+5*6  =  31  =  3/1-2+5*6
4/1-2+5*6  =  32  =  4/1-2+5*6
Hay  16  números para los cuales el algoritmo de Fuerza Bruta es más rápido, y so
n los siguientes:  [-17, -9, -6, -4, 2, 3, 4, 5, 6, 8, 10, 11, 12, 15, 22, 26]
Hay  37  números para los cuales el algoritmo de Backtracking es más rápido, y so
n los siguientes:  [-27, -25, -24, -23, -19, -16, -15, -14, -12, -11, -10, -8, -
7, -5, -3, -2, -1, 0, 1, 7, 9, 13, 14, 16, 17, 18, 19, 20, 21, 23, 24, 25, 27, 2
8, 29, 31, 32]
Hay  0  números para los cuales los algoritmo de Backtracking y Fuerza Bruta son
```

```
igual de rápidos, y son los siguientes:  []
El algoritmo de Backtracking con 0.026987127603824088s es más rápido en promedio
que el de Fuerza Bruta con 0.029478077566207777s.
Diferencia en el tiempo promedio: 1.0923014112117186
```

Por tanto, el algoritmo de Backtracking continua siendo más rápido en promedio y, además, también es más rápido en la mayoría de los casos.

5 números

In [ ]:
```
lista_numeros=[1,2,3,4,5]
comprobacionTiempos()
```

```
2/1+3-4*5  =  -15  =  2/1+3-4*5
1-3*5+4/2  =  -12  =  1-3*5+4/2
2/1-3*5+4  =  -9  =  2/1-3*5+4
1+3-4/2*5  =  -6  =  1+3-4/2*5
2/1-3*4+5  =  -5  =  2/1-3*4+5
3/1-2*5+4  =  -3  =  3/1-2*5+4
1/2*4+3-5  =  0  =  1/2*4+3-5
1+3/2*4-5  =  2  =  1+3/2*4-5
4/1-2*3+5  =  3  =  4/1-2*3+5
1/2*4-3+5  =  4  =  1/2*4-3+5
2/1*3+4-5  =  5  =  2/1*3+4-5
1*3-4/2+5  =  6  =  1*3-4/2+5
2/1*3-4+5  =  7  =  2/1*3-4+5
1-3+4/2*5  =  8  =  1-3+4/2*5
2/1+3*4-5  =  9  =  2/1+3*4-5
2/1*4-3+5  =  10  =  2/1*4-3+5
2/1*5-3+4  =  11  =  2/1*5-3+4
3-1+4/2*5  =  12  =  3-1+4/2*5
2/1+3*5-4  =  13  =  2/1+3*5-4
1+3*5-4/2  =  14  =  1+3*5-4/2
3/1*4-2+5  =  15  =  3/1*4-2+5
3*5-1+4/2  =  16  =  3*5-1+4/2
3/1*5-2+4  =  17  =  3/1*5-2+4
2/1-3+4*5  =  19  =  2/1-3+4*5
3/1-2+4*5  =  21  =  3/1-2+4*5
Hay  22  números para los cuales el algoritmo de Fuerza Bruta es más rápido, y so
n los siguientes:  [-15, -9, -5, -3, 0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 1
5, 16, 17, 19, 21]
Hay  3  números para los cuales el algoritmo de Backtracking es más rápido, y son
los siguientes:  [-12, -6, 14]
Hay  0  números para los cuales los algoritmo de Backtracking y Fuerza Bruta son
igual de rápidos, y son los siguientes:  []
El algoritmo de Fuerza Bruta con 0.0070886735599196985s es más rápido en promedio
que el de Backtracking con 0.009171895200270229s.
Diferencia en el tiempo promedio: 0.7728690096361815
```

Podemos ver que para el caso en que hay 5 números el algoritmo de Fuerza Bruta es más eficaz que el de Backtracking en promedio, siendo el único tamaño del problema donde esto pasa. Además, también es más eficaz para una mayor cantidad de casos, siendo, de nuevo, el único tamaño del problema donde sucede esto.

También observamos como la diferencia en el tiempo promedio va creciendo a medida que la cantidad de números se hace más grande. Por lo que cabría esperar que el único caso en el que el algoritmo de Fuerza Bruta es más eficiente es para n=5

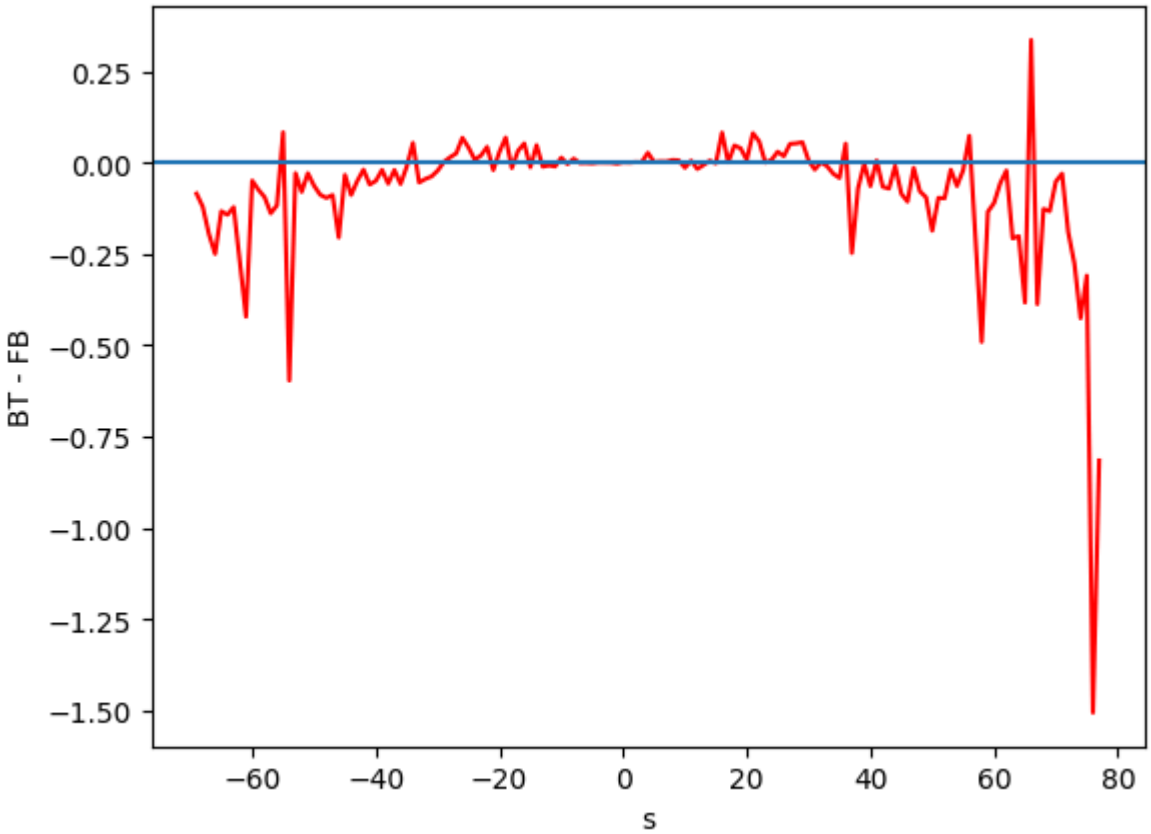
Otra pregunta que cabría plantearse es si el ahorro de tiempo es igual en todo el rango de números.

In [45]:
```
import matplotlib.pyplot as plt

timeDiference=[]
for i in range(len(timeBT)):
        timeDiference.append(timeBT[i]-timeFB[i])

x = list(range(-69, 78))  # de -69 a 77

plt.plot(x, timeDiference,'r')
plt.axhline(0)
plt.xlabel("s")
plt.ylabel("BT - FB")
plt.show()
```



Como podemos ver en la gráfica, en los números más ceracanos al 0 el algoritmo de fuerza bruta es ligeramente más eficiente, mientras que, cuanto más se aleja del 0 mejor se comporta e algoritmo de Backtraking en comparación con el de Fuerza Bruta. Este resultado está en linea con lo visto en las reducciones de las cantidades de números ya que cuantos menos números cogemos, menor es el rango de números que es capaz de alcanzar.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

In [ ]:
```
import random

for s in range(20):
  n = random.randint(-70, 80)
  print(str(n)+": ",operaciones_para(n))
```

Aplica el algoritmo al juego de datos generado

Respuesta

In [ ]:
```
40:  1+6*7-9/3
26:  1+3*9-4/2
14:  1+2*8-9/3
26:  1+3*9-4/2
45:  2/1-5+6*8
69:  2/1-5+8*9
-54:  3/1+6-7*9
-8:  1-2*6+9/3
27:  1+4*7-6/3
33:  1+4*9-8/2
25:  1+3*9-6/2
-22:  1-3*9+8/2
-3:  1+2-3/4*8
-62:  3/1+7-8*9
69:  2/1-5+8*9
-7:  1+2*3/6-9
-55:  2/1+6-7*9
-65:  2/1+5-8*9
-26:  1-5*6+9/3
-26:  1-5*6+9/3
```

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Respuesta

Un posible avance en el problema podría ser encontrar más heurísticas que pudiesen mejorar el algoritmo de Backtracking.

Referencias

- https://www.delftstack.com/es/howto/python/python-elapsed-time/

Uso de la IA

- Se ha usado ChatGPT con el siguiente prompt: Simplifica (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+1+M(5+3 (3M+N+9)+1+N((4+3 5)+1+M(5+3 5)+3+1))+3+1))+3+1))+3+1))+3+1)

¿Puedes escribírmelo en Latex?